Urban Soles: Project & Theme Documentation

Oct 31, 2026 · @mishkat

Project Overview

Urban Soles is a full stack sneaker e-commerce store for Pakistan, built with Next.js, Node.js and MongoDB, with cash on delivery (COD) as the payment method.

Item	Detail
Brand name	Urban Soles (written URBAN**SOLES**, with SOLES in orange)
Niche	Sneakers for Men, Women, Sports and Casual wear
Tagline	Sneakers for every street
Target users	Students and young professionals in Pakistan, 18 to 35 years
Payment	Cash on delivery (Stripe is not available in Pakistan)
Theme	Light, minimal, premium (white and grey with one orange accent)
Main goal	A deployed portfolio project that proves full stack skills for junior developer jobs

Theme and Brand

The theme is a clean light design: white backgrounds, light grey sections, near-black text and a single orange accent used only for actions and highlights.

Role	Name	Hex	Where to use it
Background	White	#FFFFFF	Page background, navbar
Surface	Light grey	#F5F5F5	Hero, product image boxes, footer
Text	Ink	#111111	Headings and body text
Muted text	Grey	#6B7280	Categories, captions, footer text

Role	Name	Hex	Where to use it
Accent	Orange	#F97316	Primary buttons, prices, links on hover
Accent dark	Dark orange	#EA580C	Button hover state
Border	Line grey	#E5E7EB	Card borders, dividers
Success	Green	#16A34A	In stock, order delivered
Error	Red	#DC2626	Form errors, out of stock

Brand rules:

- Write the logo as URBAN in ink and SOLES in orange, in Poppins Bold.
- Use orange for at most one main action per screen.
- Keep the tone short, confident and friendly. Example: "Sneakers for every street."
- Use a fictional brand only. Never use real brand names or logos such as Nike or Adidas.

Typography and Spacing

Poppins is used for headings and the logo, and Inter is used for everything else; both load free through next/font/google.

Element	Font	Tailwind classes
Logo	Poppins Bold	font-heading font-bold text-xl
Hero heading	Poppins Bold	font-heading font-bold text-4xl md:text-6xl
Section heading	Poppins SemiBold	font-heading font-semibold text-2xl
Product name	Inter SemiBold	font-semibold
Body text	Inter Regular	text-base
Small label	Inter Regular	text-xs uppercase tracking-wide text-muted

Spacing and shape rules:

- Page width: max-w-6xl mx-auto px-4.
- Section spacing: py-12 on desktop, py-8 on mobile.
- Grid gaps: gap-6 for product grids, gap-4 for category tiles.
- Buttons and the cart button use rounded-full; cards and images use rounded-xl.
- Product images use a 4:5 ratio (aspect-[4/5]) with object-cover.
- Shadows only on hover (hover:shadow-lg). Keep the rest flat.

- Design mobile first: write the small-screen style first, then add sm:, md: and lg: styles.

UI Components

Eight reusable components cover the whole store, and each one follows the theme above.

Component	File	Design rule
Navbar	components/Navbar.js	Sticky, white with blur, h-16, border-b; logo left, links centre, Login and Cart right
Footer	components/Footer.js	bg-surface, border-t, small muted text
Product card	components/ProductCard.js	4:5 grey image box, zoom on hover, category label, name, orange price
Primary button	Tailwind classes	bg-brand text-white rounded-full px-8 py-3, hover:bg-brand-dark
Secondary button	Tailwind classes	bg-ink text-white rounded-full, hover:bg-brand
Hero	app/page.js	bg-surface, two columns: text and call to action left, image right
Category tile	app/page.js	Bordered box, hover:border-brand hover:text-brand
Form input	components/Input.js	border-line rounded-lg px-4 py-3, focus:border-brand, red message under it on error

States every component needs:

- Loading: grey skeleton boxes (animate-pulse) while data loads.
- Empty: a friendly message, for example "Your cart is empty" with a Shop Now button.
- Error: a red message with a retry option.
- Feedback: a small toast after actions such as "Added to cart".

Pages and Features

The store has 8 customer pages and 3 admin pages; build the must-have items first and add the rest only if time allows.

Page	Route	Access	What it does	Priority
Home	/	Public	Hero, categories, featured sneakers	Must
Shop	/products	Public	Grid, search, category filter, price sort	Must
Product detail	/products/[id]	Public	Image, price, size picker, Add to Cart	Must
Cart	/cart	Public	Items, quantity, remove, total	Must
Signup and Login	/signup, /login	Public	Forms, validation, JWT session	Must
Checkout	/checkout	Logged in	Name, phone, address, place COD order	Must
My Orders	/orders	Logged in	Past orders and their status	Good
Admin dashboard	/admin	Admin only	Total orders, total products	Good
Admin products	/admin/products	Admin only	Add, edit, delete products with image upload	Must
Admin orders	/admin/orders	Admin only	View orders, change status	Good

Bonus features (only after everything above works): product reviews, wishlist, order confirmation email, pagination, dark mode.

What recruiters check: a live link, protected routes (checkout needs login), separate user and admin roles, clean code, and a README with screenshots.

Tech Stack

The whole store runs on Next.js with MongoDB, and every tool in the stack has a free tier.

Layer	Tool	Why
Framework	Next.js (App Router)	Frontend and backend in one project, server components for fast pages
Styling	Tailwind CSS	Fast, consistent, responsive design

Layer	Tool	Why
Database	MongoDB Atlas (free M0)	Flexible documents, free hosting
ODM	Mongoose	Models and validation for MongoDB
Auth	JWT + bcryptjs	Token login and hashed passwords
Images	Cloudinary	Free image upload and hosting for admin products
State	React Context + localStorage	Cart survives page refresh
Deploy	Vercel	Free hosting with a live link

Data rule: server components read from MongoDB directly for display pages (shop, product detail, orders list). Write actions (signup, login, orders, admin changes) go through REST API routes, so the project can honestly list REST APIs on a resume.

Database Models

Three MongoDB collections hold everything: User, Product and Order.

User

Field	Type	Notes
name	String	Required
email	String	Required, unique
password	String	Hashed with bcryptjs, never stored as plain text
role	String	"user" or "admin", default "user"
createdAt	Date	Added automatically

Product

Field	Type	Notes
name	String	Required
description	String	Required
price	Number	In Pakistani rupees (Rs.)

Field	Type	Notes
category	String	Men, Women, Sports or Casual
sizes	[Number]	For example 40, 41, 42
image	String	Cloudinary URL
stock	Number	Default 0
featured	Boolean	Shown on the home page

Order

Field	Type	Notes
user	ObjectId	Reference to User
items	Array	Each item has product, name, price, size, quantity
totalPrice	Number	Calculated on the server, not trusted from the browser
address	Object	fullName, phone, city, street
paymentMethod	String	"COD"
status	String	pending, shipped or delivered
createdAt	Date	Added automatically

Folder Structure and API Routes

Pages live in app/, reusable UI in components/, database code in models/ and lib/, and every write action has its own REST route.

```
urban-soles/
├── app/
|   ├── layout.js            (fonts, Navbar, Footer)
|   ├── page.js              (home)
|   ├── globals.css          (theme colours)
|   ├── products/
|   |   ├── page.js          (shop)
|   |   └── [id]/page.js     (product detail)
|   ├── cart/page.js
|   ├── checkout/page.js
|   ├── orders/page.js
```

```
|   ├── login/page.js
|   ├── signup/page.js
|   ├── admin/
|   |   ├── page.js
|   |   ├── products/page.js
|   |   └── orders/page.js
|   └── api/
|       ├── auth/signup/route.js
|       ├── auth/login/route.js
|       ├── products/route.js
|       ├── products/[id]/route.js
|       └── orders/route.js
├── components/              (Navbar, Footer, ProductCard, Input)
├── models/                  (User.js, Product.js, Order.js)
├── lib/                     (db.js, auth.js)
├── scripts/seed.js          (inserts the 10 sample sneakers)
└── .env.local               (secrets, never pushed to GitHub)
```

Method	Endpoint	Access	Purpose
POST	/api/auth/signup	Public	Create account
POST	/api/auth/login	Public	Log in, return JWT
GET	/api/products	Public	List products with search and filters
GET	/api/products/[id]	Public	Get one product
POST	/api/products	Admin	Add product
PUT	/api/products/[id]	Admin	Edit product
DELETE	/api/products/[id]	Admin	Delete product
POST	/api/orders	Logged in	Place a COD order
GET	/api/orders	Logged in	Own orders (admin sees all)
PUT	/api/orders/[id]	Admin	Update order status

Setup Guide

Setup takes about an hour: create the project, add the theme, connect MongoDB and push to GitHub.

1. Install Node.js (LTS), VS Code and Git.
2. Create the project and install packages:

```
npx create-next-app@latest urban-soles
cd urban-soles
npm install mongoose bcryptjs jsonwebtoken
```

Choose Tailwind CSS: Yes and App Router: Yes.

3. Add the theme to app/globals.css (Tailwind v4 syntax):

```
@import "tailwindcss";

@theme inline {
  --color-brand: #F97316;
  --color-brand-dark: #EA580C;
  --color-ink: #111111;
  --color-muted: #6B7280;
  --color-surface: #F5F5F5;
  --color-line: #E5E7EB;
  --font-heading: var(--font-poppins);
  --font-sans: var(--font-inter);
}

body {
  background: #ffffff;
  color: #111111;
}
```

4. Create a free MongoDB Atlas cluster (M0), add a database user, allow network access, and copy the connection string.
5. Create .env.local in the project root:

```
MONGODB_URI=mongodb+srv://USERNAME:PASSWORD@cluster.mongodb.net/urbansoles
JWT_SECRET=a_long_random_secret_string
```

6. Start the app and push to GitHub:

```
npm run dev
git init
git add .
git commit -m "Initial setup"
```

Never push .env.local. The default Next.js .gitignore already excludes it.

Build Roadmap

The store can be finished in about 15 days in 7 phases; tick each item as you complete it.

Phase 1: Setup and theme (Day 1)

- [ ] Create the Next.js project and install packages
- [ ] Add theme colours and fonts, Navbar and Footer
- [ ] Build the Home page with dummy data

Phase 2: Database and products (Days 2 to 4)

- [ ] Connect MongoDB (lib/db.js) and create the three models
- [ ] Write scripts/seed.js and insert the 10 sneakers
- [ ] Shop page and product detail page read from the database
- [ ] Add search, category filter and price sort

Phase 3: Authentication (Days 5 and 6)

- [ ] Signup and login API routes with bcrypt and JWT
- [ ] Login and signup pages with validation
- [ ] Protect checkout and orders pages

Phase 4: Cart (Days 7 and 8)

- [ ] Cart context with add, remove and quantity
- [ ] Save the cart in localStorage
- [ ] Cart page with total price and live Navbar count

Phase 5: Orders (Days 9 and 10)

- [ ] Checkout form with address
- [ ] POST /api/orders (price calculated on the server)
- [ ] My Orders page

Phase 6: Admin panel (Days 11 and 12)

- [ ] Admin role check on pages and API routes
- [ ] Add, edit and delete products with Cloudinary upload
- [ ] View orders and change their status

Phase 7: Polish and deploy (Days 13 to 15)

- [ ] Loading, empty and error states, toast messages
- [ ] Test on a phone-sized screen
- [ ] Deploy to Vercel and add environment variables
- [ ] Write the README with screenshots and the live link

Resume and README Tips

Once the store is live, use these ATS-friendly bullet points on your resume, and change them to match what you actually built.

Resume project entry

Urban Soles | Full Stack E-commerce Store | Next.js, React, Node.js, MongoDB, Tailwind CSS | [Live Demo] | [GitHub]

- Built a full stack sneaker e-commerce web application using Next.js, Node.js and MongoDB with a fully responsive UI.
- Developed REST APIs for authentication, products and orders, with JWT login and bcrypt password hashing.
- Implemented a shopping cart, COD checkout and order history for logged-in users.
- Created an admin dashboard with role-based access to manage products, images and order status.
- Deployed the application on Vercel with MongoDB Atlas and Cloudinary.

README checklist

- [ ] Project name, one-line description and live link at the top
- [ ] 3 to 5 screenshots (home, shop, cart, admin)
- [ ] Features list and tech stack
- [ ] How to run it locally (install, .env variables, seed script, start)
- [ ] Demo admin login for recruiters (a test account only, never your real password)

Before you share the link

- [ ] Replace placeholder images with real sneaker photos from Unsplash or Pexels
- [ ] Check every page on a phone screen
- [ ] Confirm .env.local is not on GitHub
- [ ] Pin the repository on your GitHub profile and add the live link to your resume and LinkedIn